Generazione dei certificati

Kubernetes richiede dei certificati con relative chiavi per le connessioni autenticate tra i suoi componenti interni e con l'esterno. Docker Desktop si occupa di generare questi certificati server e client per i principali servizi interni: kubelet (node manager), service account management, frontproxy, API server e i componenti etcd saranno quindi autenticati e potranno comunicare tra loro in sicurezza.

Configurazione del cluster

Docker Desktop installa Kubernetes utilizzando `kubeadm`, pertanto deve creare il runtime `kubeadm` e la configurazione a livello di cluster. Ciò include la configurazione di rete del cluster, i certificati, l'endpoint del nodo e così via. Anche tutti i nomi che verranno utilizzati dagli URL delle risorse che utilizzeremo per far comunicare i diversi componenti saranno sempre impostati tramite Docker Desktop, mentre l'endpoint globale del cluster utilizzerà il nome DNS `https://kubernetes.docker.internal:6443`. La porta 6443 è infatti la porta predefinita a cui è associato il nodo principale di Kubernetes.

Installazione dei componenti principali

All'interno di Docker Desktop, un processo di gestione chiamato *Lifecycle service* si occupa dell'avvio di servizi come il demone Docker e della notifica del loro cambiamento di stato. Per esempio, una volta generati i certificati nel primo passaggio e completata la configurazione di Kubernetes nel secondo, viene inviata una richiesta al servizio Lifecycle per installare e avviare Kubernetes. La richiesta contiene i certificati necessari per l'installazione. Questo servizio inizia quindi a estrarre tutte le immagini dei componenti interni di Kubernetes da Docker Hub e a scaricarle per il successivo avvio. Queste immagini contengono file binari come `kubelet`, `kubeadm`, `kubectl`, `crictl` ecc. che vengono estratti e inseriti nel path `/usr/bin`.

Avvio del cluster

Una volta che questi binari sono attivi e funzionanti, e i file di configurazione sono stati aggiunti nei percorsi corretti, il servizio Lifecycle esegue il comando `kubeadm init` per inizializzare il cluster e quindi avviare il processo kubelet. Poiché si tratta di un cluster con un singolo nodo, viene eseguita solo un'istanza kubelet. Il servizio Lifecycle attende quindi che i Pod di sistema come `coredns`, `kube-controller-manager` e `kube-apiserver` siano in esecuzione per notificare al servizio principale di Docker Desktop che Kubernetes è stato avviato.

Configurazione della gestione di rete e della persistenza

Una volta avviati i servizi interni di Kubernetes, Docker Desktop avvia l'installazione di controller aggiuntivi come lo *storage-provisioner* e *vpnkit-controller*. I loro ruoli riguardano la persistenza dello stato dell'applicazione a seguito di possibili riavvii/aggiornamenti e come accedere alle applicazioni una volta che queste sono state installate nel cluster. Una

volta che questi controller sono attivi e in esecuzione, il cluster Kubernetes è completamente operativo e possiamo eseguire i comandi `kubectl` per distribuire le applicazioni.

Configurazione della dashboard

Sarebbe davvero bello se Docker Desktop includesse una GUI per Kubernetes pronta all'uso, ma possiamo ottenere un'esperienza simile utilizzando una *Kubernetes Dashboard*. Tramite il suo progetto ufficiale (repository del progetto: `https://github.com/kubernetes/dashboard`), sempre frutto del lavoro del team di lavoro di Kubernetes, viene fornita un'interfaccia utente Web per la gestione del cluster, per garantire un modo semplice che permetta di visualizzare una panoramica sullo stato del cluster e persino per la modifica e l'esecuzione dei container e delle altre risorse che vedremo.

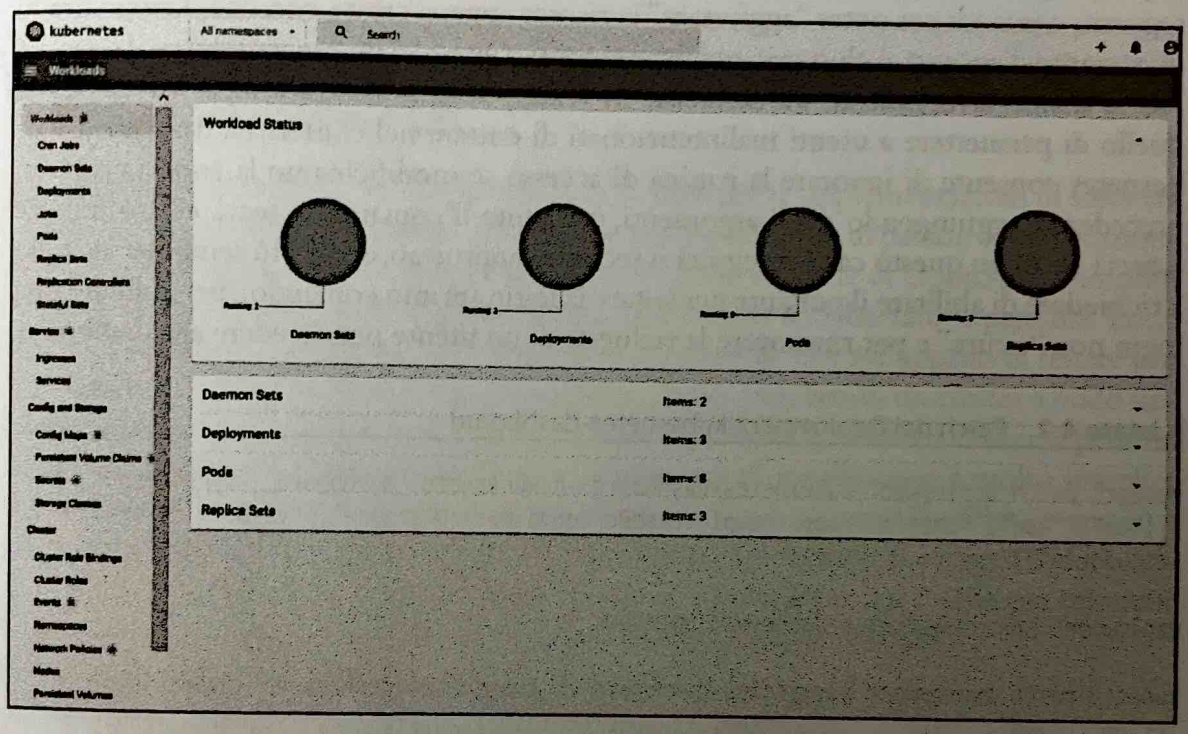

Figura 4.8 Pagina principale della dashboard di Kubernetes.

Per installare Kubernetes Dashboard, apri un terminale ed esegui quanto segue:

Listato 4.1 Configurazione della dashboard

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/v2.7.0/aio/
deploy/recommended.yaml

>>>
namespace/kubernetes-dashboard created
serviceaccount/kubernetes-dashboard created
service/kubernetes-dashboard created
secret/kubernetes-dashboard-certs created
secret/kubernetes-dashboard-csrf created
secret/kubernetes-dashboard-key-holder created
configmap/kubernetes-dashboard-settings created
```

```
role.rbac.authorization.k8s.io/kubernetes-dashboard created
clusterrole.rbac.authorization.k8s.io/kubernetes-dashboard created
rolebinding.rbac.authorization.k8s.io/kubernetes-dashboard created
clusterrolebinding.rbac.authorization.k8s.io/kubernetes-dashboard created
deployment.apps/kubernetes-dashboard created
service/dashboard-metrics-scraper created
deployment.apps/dashboard-metrics-scraper created
```

L'output dovrebbe essere simile a quanto riportato; se adesso questi comandi sembreranno confusi e poco chiari, non c'è da preoccuparsi: tutto sarà più chiaro quando entreremo nel vivo di questa tecnologia. Per ora, basti sapere che il comando precedente ha preso un file YAML e l'ha dato in pancia al motore di Kubernetes, che ha creato una serie di oggetti a partire dalla loro definizione.
Dal momento che non ci intendiamo ancora di utenti, dovremo anche fare qualche step in più per poter accedere alla dashboard da browser senza alcuna credenziale: eseguiamo i seguenti comandi per poter "aggiustare" le risorse appena create, così che siano in grado di garantirci l'accesso al cluster in maniera insicura. Questo è un procedimento da seguire *solo* se stiamo lavorando in un ambiente di sviluppo come il nostro laptop: il rischio è quello di permettere a utenti malintenzionati di entrare nel cluster! La dashboard Kubernetes consente di ignorare la pagina di accesso se modifichiamo la risorsa creata in precedenza aggiungendo degli argomenti, o tramite il comando `kubectl edit` o tramite `kubectl patch`. In questo caso, si seguirà il secondo approccio, che è più semplice: si andrà a richiedere di abilitare il pulsante per saltare la login (primo comando), per abilitare una login non "sicura" e per rimuovere la richiesta di un utente per accedere alla dashboard.

Listato 4.2 Patch del Deployment kubernetes-dashboard

```
kubectl patch deployment kubernetes-dashboard -n kubernetes-dashboard --type 'json' -p
'[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--enable-
skip-login"}]'
>>>
deployment.apps/kubernetes-dashboard patched

kubectl patch deployment kubernetes-dashboard -n kubernetes-dashboard --type 'json' -p
'[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--enable-
insecure-login"}]'
>>>
deployment.apps/kubernetes-dashboard patched

kubectl patch deployment kubernetes-dashboard -n kubernetes-dashboard --type 'json' -p
'[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--disable-
settings-authorizer"}]'
>>>
deployment.apps/kubernetes-dashboard patched
```

L'ultimo step permette di far sì che tutti i progetti che useremo siano visibili attraverso questa dashboard: assegnamo alla risorsa che ha installato la dashboard e di cui assumeremo l'identità i permessi massimi per agire all'interno del cluster:

Listato 4.3 Abilitazione dei permessi per tutti i namespace

```
kubectl delete clusterrolebinding kubernetes-dashboard
>>>
```

Cloud Controller Manager

Ultimo, ma non per importanza, il CCM: quando Kubernetes viene distribuito in ambienti cloud, questo controller funge da ponte tra le API della piattaforma utilizzata e il cluster. Kubernetes può essere distribuito in molti ambienti diversi e può interagire con vari provider per comprendere e gestire lo stato delle risorse nel cluster. In questo modo i componenti principali di Kubernetes possono funzionare in modo indipendente e consentire ai fornitori di servizi cloud di integrarsi con Kubernetes utilizzando i relativi plugin. L'integrazione del controller consente al cluster Kubernetes di eseguire il provisioning di risorse cloud come istanze (per aggiungere dei nodi), Load Balancer (per la comunicazione di rete) e volumi di archiviazione (per i volumi persistenti) in maniera trasparente. Questo infatti è stato originariamente creato per consentire di sviluppare Kubernetes indipendentemente dall'implementazione dello specifico cloud provider; in altre parole, funge da collante per consentire a Kubernetes di interagire con i provider e le relative funzionalità, mantenendo al contempo costrutti relativamente generici al proprio interno.

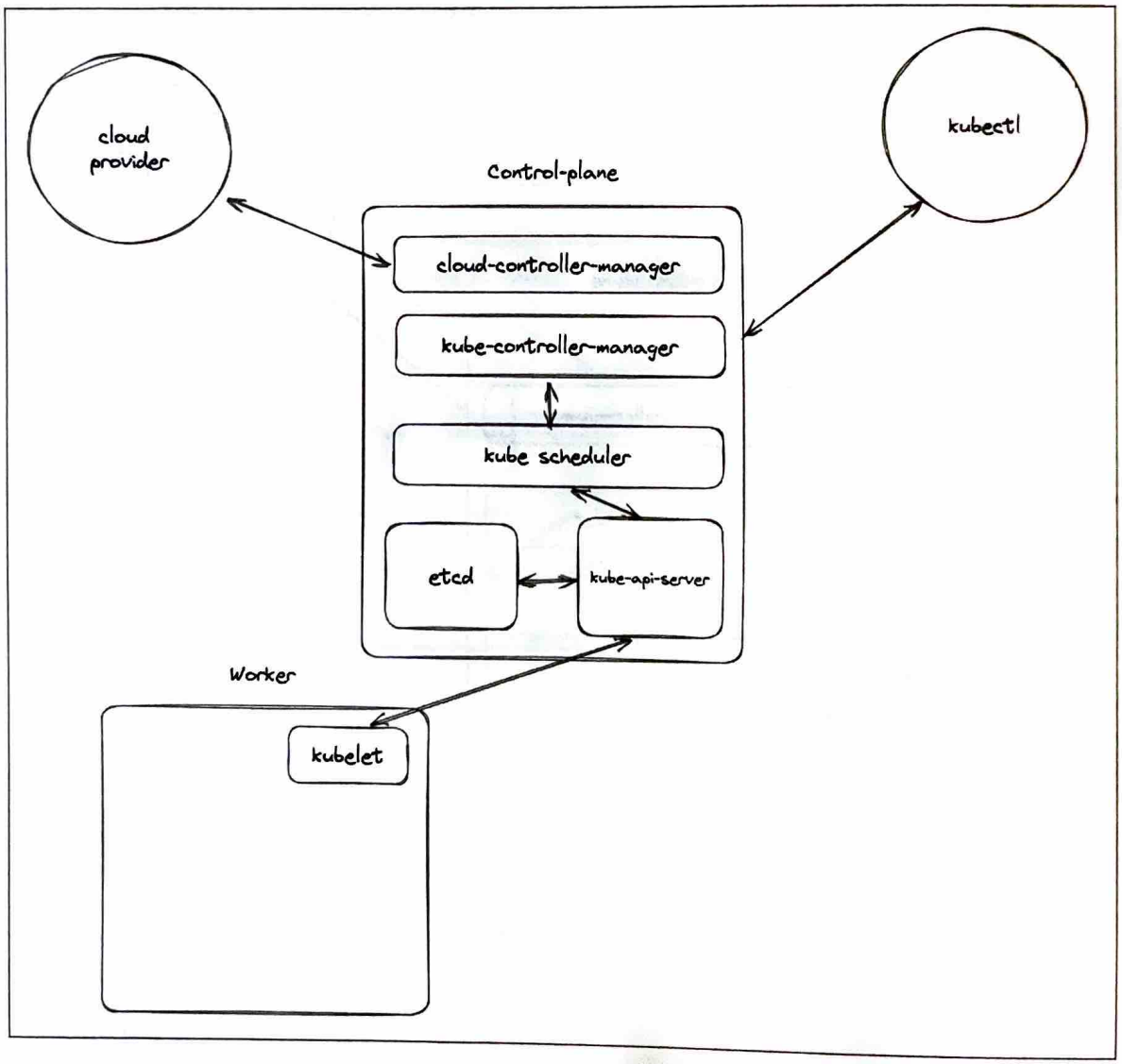

Figura 3.7 Funzionamento del componente cloud-controller-manager.

Ciò consente a Kubernetes di aggiornare le informazioni sullo stato in base alle informazioni raccolte dal provider, di adattare le risorse cloud quando sono necessarie modifiche nel sistema e di creare e utilizzare servizi aggiuntivi per soddisfare i requisiti di lavoro inviati al cluster. Il CCM ci tornerà particolarmente utile quando parleremo di Kubernetes on cloud, verso la fine del manuale, e di come questo possa essere utilizzato attraverso diversi cloud provider.

Componenti del worker

Ora che abbiamo una prima panoramica delle risorse che gestiscono attivamente il cluster, parliamo dei componenti che si occupano degli applicativi, e che risiedono nei nodi worker. Questi sono infatti fondamentali per far sì che l'applicazione funzioni correttamente e sia fruibile anche da un utente esterno!

kubelet

Il punto di contatto principale per ciascun nodo con il gruppo cluster è un piccolo servizio chiamato `kubelet`. `kubelet` è un agente che viene eseguito su ogni nodo worker del cluster come demone, gestito da systemd. Per dirla semplicemente, è responsabile di quanto segue: creazione, modifica ed eliminazione dei container per il Pod, gestione del loro avvio e della loro corretta esecuzione, dei relativi volumi e della raccolta dei dati dello stato del nodo e del Pod. `kubelet` è anche un controller che verifica le modifiche relative ai Pod e utilizza l'ambiente di esecuzione dei container del nodo per effettuare il pull delle immagini e tutte le attività che riguardano l'applicazione.
Come accennato in precedenza, `kubelet` comunica con i nodi control-plane per autenticarsi con il cluster e ricevere le istruzioni per poter operare; la definizione delle risorse con cui deve operare avviene per mezzo di un file in formato YAML, che viene anche definito *manifest*, il quale definisce l'oggetto e i parametri operativi di ciò che deve creare e/o gestire. Il processo `kubelet` si assume quindi la responsabilità di mantenere lo stato delle risorse coerente a quanto descritto nel database di etcd.

Container Runtime

Se hai mai sentito parlare di JRE, o Java Runtime, allora avrai un'idea di che cosa rappresenti questo componente: nel caso di Java si tratta del software richiesto per eseguire programmi scritti tramite questo linguaggio di programmazione su un qualsiasi host. Allo stesso modo, è necessario un ambiente di esecuzione che, tramite del software, sia in grado di gestire e avviare dei container. In effetti, il primo componente che ogni nodo worker deve avere è un *container runtime*.
Il container runtime è responsabile dell'avvio e della gestione dei container, delle applicazioni incapsulate in un ambiente operativo relativamente isolato ma leggero. Ogni unità di lavoro sul cluster è implementata come uno o più container che devono essere distribuiti; il container runtime su ciascun nodo è il componente che esegue i container definiti nei carichi di lavoro inviati al cluster; viene eseguito su tutti i nodi nel cluster Kubernetes ed è responsabile del pull delle immagini dai registry come DockerHub o simili, dell'esecuzione dei container, dell'allocazione e dell'isolamento delle risorse per i

container e della gestione dell'intero ciclo di vita di un container su un host. In questo dominio, esistono due concetti chiave.

- *Container Runtime Interface (CRI)* è quel componente che rappresenta un insieme di API per consentire a Kubernetes di interagire con diversi ambienti di esecuzione per container. In questo modo, è possibile utilizzare in modo intercambiabile diversi strumenti per container con Kubernetes, visto che definisce un'API per la creazione, l'avvio, l'arresto e l'eliminazione di container, nonché per la gestione di immagini e la loro comunicazione. Per fare un esempio, Docker Engine è un container runtime che rispetta gli standard definiti all'interno di CRI.
- *Open Container Initiative (OCI)* è invece un insieme di standard per i formati e i tempi di esecuzione dei container.

Kubernetes supporta più container runtime, e ciò significa che tutti questi implementano l'interfaccia CRI ed espongono le API per l'esecuzione e la gestione delle immagini.

Open Container Initiative

L'OCI è un progetto formatosi grazie alla Linux Foundation per definire uno standard per tutte le risorse che gravitano attorno al mondo dei container. L'OCI è stato lanciato il 22 giugno 2015 da Docker, CoreOS e altri leader nel settore dei container.